

AI WEATHER WISE API

Short Project Documentation

1. Abstract

AI Weather Wise API is a REST-based application that provides weather information to websites and mobile applications. The system receives a location such as a city name or latitude/longitude, communicates with an external weather service, and returns useful weather data in JSON format.

The API can provide details such as temperature, feels-like temperature, humidity, pressure, wind speed, visibility, and weather condition. An optional recommendation module can convert these values into simple user-friendly suggestions.

2. Introduction

Weather information is useful for travel, outdoor activities, agriculture, transportation, events, and daily planning. Instead of connecting every application directly to an external weather provider, the Weather Wise API acts as a common backend layer. This makes the weather service reusable and easier to maintain.

The project can use a provider such as OpenWeather. Its API documentation describes current-weather and forecast services and JSON-based responses. The exact endpoints and available features depend on the selected provider and plan.

3. Problem Statement

- Applications need quick and reliable access to weather information.
- Direct integration with external APIs can increase frontend complexity.
- API keys must be protected from exposure in client-side code.
- Raw weather data may need to be simplified for end users.

4. Objectives

- Develop a reusable REST weather API.
- Retrieve current weather information using a weather provider.
- Return simple and structured JSON responses.
- Handle invalid requests, API errors, and rate limits.
- Secure the external weather API key on the server.
- Provide a foundation for weather-based recommendations.

5. System Design

The system follows a simple three-layer architecture:

```
User / Client Application
↓
AI Weather Wise API
(Node.js + Express)
↓
External Weather Provider
↓
JSON Data
↓
Process & Format
↓
Client Response
```

6. Technology Requirements

Component	Technology
Frontend	HTML/CSS/JavaScript or React
Backend	Node.js + Express
Data Format	JSON
Weather Service	OpenWeather or equivalent
Testing	Postman / Browser

7. API Design

The main endpoint can be designed as:

```
GET /api/weather?city=Chennai
```

Coordinate-based request:

```
GET /api/weather?lat=13.08&lon=80.27
```

A normalized response can contain location, temperature, humidity, wind speed, condition, and recommendation.

8. Sample JSON Response

```
{
  "success": true,
  "location": "Chennai",
  "temperature": 30.5,
  "humidity": 72,
  "windSpeed": 4.2,
  "condition": "Clouds",
  "advice": "Stay hydrated."
}
```

9. Basic Implementation

```
const express = require("express");
const cors = require("cors");
require("dotenv").config();

const app = express();
app.use(cors());
app.use(express.json());

app.get("/", (req, res) => {
  res.json({message: "AI Weather Wise API is running"});
});

app.listen(3000, () => console.log("Server started"));
```

10. Working Process

1. The user enters a city or location. 2. The client sends a request to the Weather Wise API. 3. The backend validates the request. 4. The backend calls the external weather provider using a server-side API key. 5. Weather data is received in JSON format. 6. Required fields are extracted and formatted. 7. Optional recommendation rules are applied. 8. The final JSON response is sent to the client.

11. Error Handling

Code	Meaning
200	Successful request
400	Invalid or missing input
401	Invalid authentication/API key
404	Location not found
429	Rate limit or quota exceeded
500	Internal server error
503	Weather service unavailable

12. Advantages

- Simple and reusable weather service.
- Centralized external API integration.
- Easy JSON-based communication.
- API key can remain protected on the backend.
- Can be integrated with websites and mobile applications.
- Can be extended with forecasts and recommendations.

13. Limitations

- Accuracy depends on the external weather provider.
- API quotas and rate limits may apply.
- Internet connection is required for live data.
- Forecast information is predictive and not guaranteed.

14. Future Enhancements

- 7-day forecast and weather alerts.
- Location-based automatic weather detection.
- Weather maps, charts, UV and air-quality information.
- AI-generated natural-language weather summaries.
- Travel recommendations based on weather conditions.

15. Conclusion

AI Weather Wise API is a practical project that demonstrates REST API development, external API integration, JSON processing, authentication, error handling, and weather-based recommendations. Its reusable architecture makes it suitable for web, mobile, travel, and other weather-aware applications.

Reference: OpenWeather API documentation and platform information.